Assignment 4: Refactoring

Upon reviewing code that the two of us implemented (whose classes particularly focus on UI elements, enemies, and objects), we discovered a total of 9 code smells that required us to refactor. Below is what we found and how we addressed them.

Code Duplication:

MainMenuScreen/render(), Header Text

Commit 7d8a0c9

Based on the menu layout, the header text changes to indicate the layout of buttons. This was initially done by setting the text and then calling the draw method for each possible header. However, by noticing that the only change between the repeated code was the header text, the repeated code could be implemented as a new method with the text as the only parameter.

MainMenuScreen/render(), How To Play

Commit 7d8a0c9

The "How To Play" layout in the main menu included duplicated text and image drawing 8 times, defining a new string each time and only changing the y position with each duplication. The solution was creating a new setHowToPlayText() method that only takes the text, image, and y position, which can be decremented in-line. This makes the code much more readable by only writing the variables for each new line, and the maintainability is improved since any rendering changes are only made in one spot.

Eraser & PencilSharpener, beforeMovementUpdate()

Commit c655195

Upon reviewing the movement logic, we found that both the Eraser and PencilSharpener classes contained the exact same nested if/else block inside their beforeMovementUpdate() method. To eliminate this code duplication, we moved the shared portions of the code (which involved the velocity calculation logic) into MobileEnemy. We then updated both subclasses to call super, allowing them to inherit the calculation while still maintaining their own attack/cooldown logic.

Long List of Method Parameters:

MenuButton constructor

Commit 6f28193

The original constructor for a MenuButton object contained 8 parameters, including width, height, position, textures, etc. Upon inspection, there were multiple parameters that never changed across implementation, or could be simplified further. The width and height was replaced with a scale factor, since all buttons in the game use a 10:1 aspect ratio. Additionally, the textures were removed from the parameters, since the only change ever made to them is the disabled texture, which was already tracked with a boolean, enabling a global texture declaration. This also centralized the test mode conditions, making the code more readable in many other areas.

Poorly Structured Code:

MainMenuScreen & GameScreen, activateButton()

Commit 6f28193

The button activation code was highly repeated across both screen classes, and was poorly structured, relying on the index of each button in the array. These code smells caused terrible maintainability and

readability, as making any layout changes required moving around the triggers of each other button, and reading the functionality based on index was difficult. To solve this, a Runnable action was added to the button object itself, allowing for predetermined functionality each button upon creation. This allows for buttons to be easily remapped and moved, and it reduces the complexity of the screen classes.

Unsafe/Unsound Constructs:

Button Texture Initialization

Commit 6f28193

In many locations throughout the code, such as the now-removed alternate constructor of GameScreen for test mode, there is a large amount of creating the same button with a null texture for testing purposes, which also requires null-checking in many more places. This involved making many accommodations with each change to the buttons to ensure proper null functionality. The solution to this issue was to move most test mode checks to the MenuButton object, containing the null handling and eliminating the duplicate constructions.

Feature Envy:

MainMenuScreen/createCenteredButton()

Commit 6f28193

The createCenteredButton method was initially a helper function for reducing the repeated text in button initialization, and was used on and off inside MainMenuScreen. Since GameScreen also had very similar uses for this method, it became an issue of either duplicating the button code, or calling it from MainMenuScreen. To eliminate the issue, the method was moved as a factory method in the MenuButton class, and the parameters and logic were modified to accommodate a wider variety of uses, allowing nearly every button creation to use this instead and creating much more cohesive and readable code.

Unjustified Use of Primitives:

Eraser & PencilSharpener

Commit 635c8ed

Both the Eraser and PencilSharpener classes used primitive integers (0, 1, 2, 3) to represent our directional states (down, up, left, right). Because these states were defined independently in each subclass, a bug appeared where the integer mappings for left and right were swapped between the two enemy types. To address this issue, we centralized the direction constants into the MobileEnemy superclass, effectively eliminating our magic numbers, and also so that inconsistencies between these subclasses wouldn't occur again.

Confusing Class Hierarchy:

Nonplayer

Commit cd0a770

The Nonplayer abstract class was acting as an unnecessary middleman in our inheritance tree. Because it contained no actual shared implementation or state of its own, forcing all non-player objects to extend it created a confusing and restrictive class hierarchy. To solve this, we converted Nonplayer into a pure interface containing methods that each of our objects share; specifically playerCollide(), updateInternal(), and render(). We then updated all objects to extend Entity directly, while implementing the Nonplayer inheritance. This provides more flexibility for future class design while also boosting readability by creating an intuitive understanding.